Brute Force Technique

1. Brute Force

- Brute Force is a problem-solving technique in which all possible solutions are checked one by one.
- It is usually the simplest approach to solve a problem.
- It does not use advanced optimization initially.
- It is useful for:
  - Understanding the problem.
  - Developing a basic solution.
  - Testing optimized solutions.
  - Solving problems with small input sizes.

Example: Pair Sum Problem

- Find whether an array contains a pair whose sum is equal to a given target.
- Check every possible pair and compare its sum with the target.

Pseudo Code:

```
for every i
    for every j > i
        if arr[i] + arr[j] == target
            pair found
```

- Time Complexity: $O(n^2)$
- Space Complexity: O(1)

---

2. Limitations of Brute Force

SUNBEAM

- May perform unnecessary calculations.
- Usually has higher time complexity.
- Becomes inefficient for large input sizes.
- Does not take advantage of the properties of the problem.

---

3. Optimization Over Brute Force

- Optimization means improving the brute-force solution to reduce:
  - Time complexity.
  - Space complexity.
  - Unnecessary operations.

Common Optimization Techniques

1. Sorting
2. Hashing
3. Two Pointer Technique
4. Sliding Window
5. Binary Search
6. Prefix Sum
7. Greedy Approach
8. Divide and Conquer
9. Dynamic Programming

SUNBEAM

---

4. Example: Pair Sum Problem

Approach 1: Brute Force

- Check every possible pair.
- Time Complexity: $O(n^2)$
- Space Complexity: O(1)

Approach 2: Hashing

- Store previously visited elements in a HashSet.
- For every element, check whether `target - arr[i]` already exists.
- If it exists, the required pair is found.
- Time Complexity: O(n) average.
- Space Complexity: O(n).

Approach 3: Sorting + Two Pointer

- Sort the array first.
- Use two pointers:
  - `left` at the beginning.
  - `right` at the end.
- If sum is smaller than target, move `left` forward.
- If sum is greater than target, move `right` backward.
- If sum equals target, the pair is found.
- Time Complexity: O(n log n), including sorting.
- Extra space depends on the sorting algorithm.

---

5. Brute Force vs Optimized Approach

Feature	Brute Force	Optimized Approach
Basic idea	Try all possibilities	Avoid unnecessary operations

SUNBEAM

Feature	Brute Force	Optimized Approach
Implementation	Usually simple	May be complex
Time Complexity	Usually higher	Usually lower
Space Usage	Often lower	May require extra space
Suitable for	Small input	Large input
Development	Good starting point	Used for efficient solutions

6. Important MCQ Points

- Brute force checks all possible candidates or solutions.
- It is generally the simplest approach.
- It is useful as a baseline solution.
- Optimization reduces unnecessary computations.
- Optimization may increase space usage to reduce time.
- Hashing can improve searching from O(n) to O(1) average.
- Two Pointer technique can reduce nested-loop solutions from $O(n^2)$ to O(n), when applicable.
- Sorting helps optimize many brute-force solutions.
- The best optimization depends on the properties of the problem.

Two Pointer Technique

1. Introduction

- Two Pointer is an algorithmic technique that uses two pointers/indices to traverse a data structure, usually an array or string.
- Both pointers can move in the same direction or in opposite directions.
- It helps avoid unnecessary nested loops.
- Commonly used for searching pairs, removing duplicates, reversing arrays and checking palindromes.

SUNBEAM

2. Types of Two Pointer Technique

A. Opposite Direction Pointers

- One pointer starts from the beginning and another from the end.
- Both pointers move towards each other.
- Commonly used with sorted arrays.

Example: Pair Sum in a Sorted Array.

- If `arr[left] + arr[right] == target`, pair is found.
- If sum is smaller than target, increment `left`.
- If sum is greater than target, decrement `right`.

B. Same Direction Pointers

- Both pointers start from the beginning.
- They move in the same direction at different speeds or positions.
- Commonly used for removing duplicates and partitioning arrays.

3. Applications

- Pair Sum in a sorted array.
- Reverse an array or string.
- Check whether a string is a palindrome.
- Remove duplicates from a sorted array.
- Detect a cycle in a linked list (Floyd's algorithm).
- Find pairs with a given difference.

4. Complexity

- Usually reduces $O(n^2)$ solutions to O(n).
- Time Complexity: O(n) in many common problems.
- Space Complexity: Usually O(1), if performed in-place.

SUNBEAM

Sliding Window Technique

1. Introduction

- Sliding Window is a technique used to process a continuous portion (subarray or substring) of an array or string.
- It maintains a window using two pointers: `left` and `right`.
- The window expands or shrinks based on problem conditions.
- It avoids recalculating results for overlapping subarrays or substrings.

2. Types of Sliding Window

A. Fixed Size Window

- Window size remains constant.
- Move the window one position at a time.
- Remove the contribution of the outgoing element and add the incoming element.

Example: Maximum sum of a subarray of size `k`.

- Calculate the sum of the first `k` elements.
- Slide the window by subtracting the outgoing element and adding the incoming element.
- Update the maximum sum.

B. Variable Size Window

- Window size changes according to a condition.
- Expand the window by moving `right`.
- Shrink the window by moving `left` when the condition is violated or when required by the problem.

Example: Longest substring without repeating characters.

- Expand the window by adding characters.

SUNBEAM

- If a duplicate character appears, shrink the window.
- Track the maximum window length.

3. Applications

- Maximum sum subarray of size K.
- Minimum size subarray with sum greater than or equal to a target (for positive elements).
- Longest substring without repeating characters.
- Count/analyze subarrays satisfying a condition.
- Maximum number of vowels in a substring of size K.

4. Complexity

- Time Complexity: Usually O(n), when each pointer moves forward at most n times.
- Space Complexity: O(1) for simple fixed-window problems; may be O(n) when a set or map is required.

Prefix Sum Technique

1. Introduction

- Prefix Sum is a technique in which cumulative sums are stored to answer range-sum queries efficiently.
- Each prefix sum represents the sum of elements from index `0` to the current index.
- It avoids repeatedly calculating the sum of the same elements.

2. Prefix Sum Construction

For an array:

```
arr = [2, 4, 1, 5, 3]
```

Prefix Sum:

SUNBEAM

```
prefix = [2, 6, 7, 12, 15]
```

Formula:

- `prefix[0] = arr[0]`
- `prefix[i] = prefix[i - 1] + arr[i]`

3. Range Sum Query

To calculate the sum from index `L` to `R`:

- If `L == 0`:

  ```
  sum = prefix[R]
  ```

- Otherwise:

  ```
  sum = prefix[R] - prefix[L - 1]
  ```

Example:

- Array: `[2, 4, 1, 5, 3]`
- Find sum from index `1` to `3`.
- `prefix[3] - prefix[0] = 12 - 2 = 10`

4. Applications

- Range sum queries.
- Subarray sum problems.
- Finding subarrays with a given sum (using prefix sums and hashing).
- 2D matrix range sum queries.
- Cumulative frequency calculations.

5. Complexity

- Prefix sum construction: O(n).

SUNBEAM

- Range sum query: O(1).
- Extra Space: O(n).
- Useful when multiple range-sum queries are performed on an array.

Difference Between Two Pointer, Sliding Window and Prefix Sum

Feature	Two Pointer	Sliding Window	Prefix Sum
Basic idea	Use two indices	Maintain a continuous window	Store cumulative sums
Pointer usage	Two pointers	Usually left and right	Not necessary
Main purpose	Efficient traversal/search	Process subarrays/substrings	Fast range-sum queries
Typical problems	Pair sum, palindrome	Fixed/variable window problems	Range sum, subarray sum
Time Complexity	Often O(n)	Often O(n)	O(n) preprocessing, O(1) range query
Extra Space	Usually O(1)	O(1) or O(n)	O(n)

SUNBEAM